

Spring MVC

P.Mathieu

IUT de Lille
<http://www.iut-a.univ-lille.fr>
 prenom.nom@univ-lille.fr

Plan

Le starter WEB

La gestion des formulaires

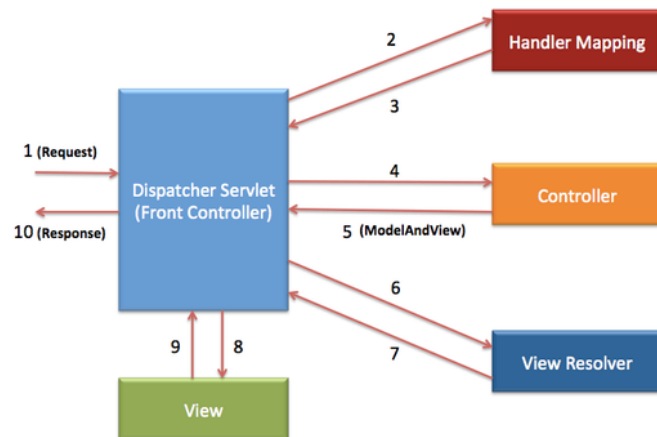
Le starter validation

Conclusion

Le starter WEB

Principe

Composant Web de Spring basé sur une architecture MVC



Le starter WEB

Couches sous-jacentes

```

HttpServletRequest
+ ServletDispatcher
+ Pattern-DAO
+ Pattern-MVC
+ ClassLoader
+ Properties
+ Jackson
+ Taglibs
+ Servlet
+ Filter
+ JPA
+ JSP
  
```



Le starter WEB

Arborescence WEB

```
tp
|-- pom.xml
|-- src
|   |-- main
|   |   |-- java
|   |   |   |-- fr
|   |   |   |   |-- but3
|   |   |   |   |   |-- tp
|   |   |   |   |   |   |-- TpApplication.java
|   |   |-- resources
|   |   |   |-- application.properties
|   |   |   |-- static
|   |   |   |-- templates
|   |-- test
|   |   |-- java
|   |   |   |-- fr
|   |   |   |   |-- but3
|   |   |   |   |   |-- tp
```

Le starter WEB

... en 111 caractères !

```
@Controller
class MonPremierContrôleur
{ @RequestMapping("/")
  @ResponseBody
  String home()
  {
    return "<h1>Hello World!</h1>";
  }
}

spring init -d=web -g=fr.but3 tp
mvn spring-boot:run
curl http://localhost:8080
```

Le starter WEB

Les Vues

Spring MVC peut être utilisé en mode REST comme en mode génération de pages HTML dynamiques. Il est auto-configuré pour les moteurs de templates suivants :

- ▶ FreeMarker
- ▶ Groovy
- ▶ Thymeleaf (default)
- ▶ Mustache.js

Dans ce cas il cherche les vues dans `src/main/resources/templates`

Le starter WEB

La Vue en JSP

L'usage de JSP n'est pas automatique. Il nécessite

- 1 Une configuration manuelle dans `application.properties` :

```
spring.mvc.view.prefix=/WEB-INF/jsp/
spring.mvc.view.suffix=.jsp
```

(voir aussi plein d'autres paramètres possibles)
- 2 L'ajout de la dépendance `tomcat-embed-jasper` au pom

```
<dependency>
  <groupId>org.apache.tomcat.embed</groupId>
  <artifactId>tomcat-embed-jasper</artifactId>
  <scope>compile</scope>
</dependency>
```

Le starter WEB

Arborescence complétée

```
tp
|-- pom.xml
|-- src
|   |-- main
|   |   |-- java
|   |   |   |-- fr
|   |   |   |   |-- but3
|   |   |   |   |   |-- tp
|   |   |   |   |   |-- TpApplication.java
|   |   |-- resources
|   |   |   |-- application.properties
|   |   |   |-- static
|   |   |   |-- templates
|   |-- webapp
|   |   |-- WEB-INF
|   |   |   |-- jsp
|-- test
|   |-- java
|   |   |-- fr
|   |   |   |-- but3
|   |   |   |   |-- tp
|   |   |   |   |-- TpApplicationTests.java
```

Le starter WEB

Modèle MVC

Le Contrôleur `MonContrôleur.java`

```
@Controller
public class MonContrôleur {
    @RequestMapping(value="/")
    public ModelAndView hello() {
        ModelAndView model = new ModelAndView();
        model.setViewName("maVue");
        model.addObject("msg", "Hello World !");
        return model;
    }
}
```

la vue `maVue.jsp`

```
<h1> ${msg} </h1>
```

- ▶ Accès via : <http://localhost:8080/>
- ▶ Peu importe le nom du contrôleur et le nom des méthodes, elles sont à usage interne.

Le starter WEB

Les contrôleurs

- ▶ On peut créer autant de contrôleurs que l'on souhaite
- ▶ On peut mettre autant de *handler methods* que l'on souhaite dans un contrôleur
- ▶ Pas de contrainte sur le nom des méthodes ou des classes.
- ▶ La valeur de retour d'une méthode annotée avec `@ResponseBody` constitue le corps de la réponse
- ▶ Sans cette annotation, la valeur de retour correspond au nom de la vue (sans l'extension)

Le starter WEB

URLMapping : par quelle(s) url la méthode est appelée ?

`@RequestMapping` : Annotation de classe ou de méthode de contrôleur

- ▶ `@RequestMapping(value="/")`
- ▶ `@RequestMapping(value={"/", "wel*come"})`
- ▶ `@RequestMapping(value={"/", "wel*come"}, method=RequestMethod.GET)`
- ▶ Le joker `*` est supporté dans les noms utilisés
- ▶ `RequestMethod` = GET (par défaut), POST (ainsi que PUT, PATCH, DELETE pour les services REST)

Le starter WEB

Cumul des RequestMapping



- L'annotation `@RequestMapping` peut être mise au niveau de la classe comme au niveau de la méthode.
- Dans ce cas les annotations se cumulent !

```
@Controller
@RequestMapping(value="/etudiant")
public class MonControlleur
{
    @RequestMapping(value="/lister")
    public ModelAndView listeDesEtudiants()
    {
        ModelAndView model = new ModelAndView();
        ....
    }
}
```

Appel : `http://localhost:8080/etudiant/lister`

Cette technique sera particulièrement utilisée pour les micro-services

Le starter WEB

URL Mapping : Raccourcis d'écriture



Depuis la version 4.3 Spring a introduit les annotations `@PostMapping` et `@GetMapping`

- `@PostMapping` est un raccourci pour `@RequestMapping(method = RequestMethod.POST)`
- `@PostMapping("/hello")`
- `@GetMapping("/hello")`

Le starter WEB

Param Binding : gestion des paramètres de l'url



`@RequestParam` est utilisé dans les paramètres des méthodes pour récupérer les paramètres dans l'URL. Ils sont automatiquement "castés" et liés aux variables Java

- `String hello(@RequestParam int id)`
- `String hello(@RequestParam("id") int idPersonne)`
- `String hello(@RequestParam(value="id",required=false) int id)`
- `String hello(@RequestParam(value="id",required=false, defaultValue="5") int id)`
- Par défaut : Value : nom de la variable. required : true. defaultValue est déclenchée si param null ou vide
- Il est possible de tout récupérer d'un coup en passant une Map comme argument : `@RequestParam Map<String,String> reqvPar`

Le starter WEB

Paramètres des gestionnaires de requêtes



De quoi j'ai besoin ? Spring me le fournit !

- La signature des *request handler methods* est très flexible !
- Présence et ordre des paramètres arbitraire
 - `HttpServletRequest` et `HttpServletResponse`
 - `HttpSession`
 - `Model` (interface) ou `ModelMap` (implementation)
 - `Principal`
 - Annotations diverses `@RequestParam`, `@InitBinder`, ...

voir [liste des paramètres](#) section 1.3.3

Le starter WEB

Rappel

L'objet `HttpServletRequest` donne accès à la plupart des objets

- ▶ `application = request.getServletContext();`
- ▶ `config = request.getServletConfig();`
- ▶ `session = request.getSession();`

Mais la philosophie de SpringBoot est plutôt :

- ▶ Evite les références directes à l'API Servlet
- ▶ Dis ce dont tu as besoin, Le FrontController te le donnera

Le starter WEB

Le modèle

- ▶ `ModelAndView` (ancien style). On crée un objet `ModelAndView`. On lui affecte la vue et le modèle. On renvoie cet objet.

```
@RequestMapping(value="/")
ModelAndView hello()
{
    ModelAndView model = new ModelAndView("maVue");
    model.addObject("msg", "Hello World !");
    return model;
}
```

- ▶ `Model` (interface) ou `ModelMap` (implémentation) Le modèle est créé par le FrontController, La méthode le modifie puis renvoie une référence à la vue sous forme de `String`.

```
@RequestMapping(value="/")
String hello(Model model)
{
    model.addAttribute("msg", "Hello World !");
    return "mavue";
}
```

Le starter WEB

Un exemple : Hello World !

Réaliser une appli web, avec

- ▶ Un seul endpoint accessible via 2 urls (`/`, `hello`)
- ▶ Un paramètre `nom` facultatif
- ▶ Si le paramètre est présent : `Hello Phil !`
- ▶ Si le paramètre est absent : `Hello world !`

`http://localhost:8080/hello`

`http://localhost:8080/hello?nom=phil`

Le starter WEB

Un exemple : Hello World !

la vue `mavue.jsp`

`Hello ${msg}`

Le Contrôleur `MonContrôleur.java`

```
@Controller
class MonContrôleur {
    @GetMapping("/{", "/hello"})
    String hello(@RequestParam(defaultValue="world", required=false) String nom
        , ModelMap model) {

        model.addAttribute("msg", nom);
        return "mavue";
    }
}
```

Plan

Le starter WEB

La gestion des formulaires

Le starter validation

Conclusion

La gestion des formulaires

Les formulaires

- Gestion du GET et du POST dans le même contrôleur
- Création d'un Pojo pour récupérer tous les paramètres saisis
- GET pour afficher le formulaire, Post pour le traiter

```
@RequestMapping(value = { "/addPersonne" }, method = RequestMethod.GET)
public String showAddPersonneForm() {
    return "addPersonne";
}

@RequestMapping(value = { "/addPersonne" }, method = RequestMethod.POST)
public String processAddPersonneForm(@RequestParam String nom,
                                     @RequestParam String prenom,
                                     Model model )
{
    Personne p = new Personne(prenom,nom);
    // traitement de cette personne
    model.addAttribute("personne",p)
    return "redirect:liesterPersonnes";
}
```

La gestion des formulaires

Gestion de la redirection

- `return "mavue"` passe par le `RequestDispatcher` pour appeler la vue avec la même requête.
- `return "redirect:mavue"` passe par le `sendRedirect` et crée donc une nouvelle requête (ce qui évite le problème du double envoi de formulaire en cas d'actualisation de la page)
- `return forward:autreRoute` passe aussi par le `RequestDispatcher` mais appelle une autre route (pas une vue).

Il n'est pas impératif de faire un `return "redirect:mavue"` après un POST dans un modèle MVC avec Spring Boot, mais c'est une bonne pratique dans la plupart des cas.

La gestion des formulaires

Auto-Binding des objets

Tout objet passé en paramètre de la méthode d'un contrôleur est automatiquement instancié et rempli à l'aide des paramètres HTTP

```
@RequestMapping(value = { "/addPersonne" }, method = RequestMethod.GET)
public String showAddPersonneForm() {
    return "addPersonne";
}

@RequestMapping(value = { "/addPersonne" }, method = RequestMethod.POST)
public String processAddPersonneForm(Personne p, Model model)
{
    // traitement de p
    model.addAttribute("personne",p)
    return "redirect:liesterPersonnes";
}
```

La gestion des formulaires

Extension du modèle

@ModelAttribute : annotation de méthode ou de paramètre

- ▶ Au niveau méthode : la méthode est systématiquement lancée avant l'exécution de tout traitement d'URL
- ▶ Au niveau paramètre : Recherche cet objet dans le modèle. S'il ne le trouve pas, il le crée et le remplit avec les paramètres Http de même nom (autoDataBinding) puis le range dans le modèle.

La gestion des formulaires

Formulaire avec @ModelAttribute

```
@RequestMapping(value={ "/addPersonne" }, method = RequestMethod.GET)
public String showAddPersonneForm() {
    return "addPersonne";
}

@RequestMapping(value={ "/addPersonne" }, method = RequestMethod.POST)
public String processAddPersonneForm(@ModelAttribute Personne p)
{
    // traitement de p
    return "redirect:liesterPersonnes";
}
```

Plan

Le starter WEB

La gestion des formulaires

Le starter validation

Conclusion

Le starter validation

La vérification des objets

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-validation</artifactId>
</dependency>
```

- ▶ Spécificité de Jakarta-Validation
- ▶ Chaque objet Spring peut être annoté pour vérification :
@size @Min, @Max, @Pattern @NotNull @NotBlank @NotEmpty @email
- ▶ Pour tester un objet passé en paramètre, il suffit de le préfixer par @valid dans la méthode qui l'utilise

Le starter validation

Un entity bean annoté



```
@Entity
@Data
public class Etudiant2 {

    private String prenom;

    @NotNull(message = "is required")
    @Size(min=1, max=10, message = "is required")
    private String nom;

    @NotNull(message = "is required")
    @Min(value = 1, message = "must be greater than or equal to 1")
    @Max(value = 5, message = "must be less than or equal to 5")
    private Integer groupe;

    @Pattern(regexp = "[a-zA-Z0-9]{5}", message = "only 5 chars/digits")
    private String codePostal;

    @NotNull(message = "is required")
    @Email(message = "Invalid email! Please enter valid email")
    private String email;
}
```

Le starter validation

La vérification des paramètres



- ▶ Le paramètre `BindingResult` permet de récupérer le résultat de cette validation
Notamment via la méthode `hasErrors()`
- ▶ `BindingResult` doit être mis juste après l'objet

```
@RequestMapping(method = RequestMethod.POST)
public String traite (@Valid @ModelAttribute("user") User user,
                    BindingResult result, Model model) {
    if (result.hasErrors()) return "leFormulaire"
    ...
    return "laSuite";
}
```

Plan



Le starter WEB

La gestion des formulaires

Le starter validation

Conclusion

Conclusion

Conclusion



Spring MVC	Spring JPA	Validation	Lombok
@Controller	@Entity	@Valid	@Data
@Repository	@Id	@NotNull	@Constructor
@Service	@GeneratedValue	@NotEmpty	@NoArgsConstructor
@RequestMapping	@Table	@Max	@ToString
@ResponseBody	@Column	@Size	@ToString.Exclude
@PostMapping	@Query	@Email	@JsonIgnore
@GetMapping	@OneToMany	@Digits	
@RequestParam	@ManyToOne	@Pattern	
@ModelAttribute		@Future	
@SessionAttribute			